

Syntetika CCIP-CCT Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

Immeas

MrPotatoMagic

May 2, 2026

Contents

1 About Cyfrin **2**

2 About This Document **2**

3 Disclaimer **2**

4 Copyright and License **2**

5 Audit Overview **3**

6 Audit Scope **3**

7 Risk Classification **4**

8 Protocol Summary **4**

8.1 Protocol Overview 4

8.2 Core Components 4

8.3 Cross-Chain Architecture (CCIP-CCT) 5

9 Attestation **5**

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Syntetika on Syntetika CCIP-CCT. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.0 dated May 2, 2026, was delivered to Syntetika, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Syntetika or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 5 days, the Cyfrin team conducted an audit on the Syntetika CCIP-CCT code provided by Syntetika. The findings consist of 1 High, 3 Medium & 13 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	Syntetika CCIP-CCT
Repository	monorepo
Commit	16a07be4bb3a...
Fix Commit	34eb00140d37...
Audit Timeline	Apr 20th - Apr 24th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	3	3	0
Low Risk	13	13	0
Informational	17	17	0
Gas Optimizations	2	2	0
Total	36	36	0

6 Audit Scope

The focus of this audit was changes to support Chainlink's CCT Standard and some design changes. The audit scope was limited to:

```
issuance/src/helpers/TokensHolder.sol
issuance/src/helpers/BlacklistableUpgradeable.sol
issuance/src/helpers/Whitelist.sol
issuance/src/minter/Minter.sol
issuance/src/token/HilBTC.sol
issuance/src/token/HilToken.sol
issuance/src/token/HilUSD.sol
issuance/src/vault/Distributor.sol
issuance/src/vault/StakingVault.sol
```

The following two contracts were initially part of the scope but were later removed from it:

```
deposit-registry/contracts/ComplianceCheckerUpgradeable.sol
deposit-registry/contracts/CompliantDepositRegistry.sol
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

Syntetika is an innovative "real yield" solution that bridges traditional Bitcoin with DeFi and sophisticated trading strategies, previously audited by Cyfrin.

8.1 Protocol Overview

Syntetika issues compliant synthetic tokens, **HilBTC** (18 decimals, pegged to Bitcoin) and **HilUSD** (6 decimals, pegged to USD), that represent user deposits of real collateral assets. These synthetic tokens accrue yield generated from off-chain Bitcoin trading strategies, with on-chain distribution managed through a set of smart contracts. The protocol is designed with institutional compliance requirements in mind: token transfers are gated through a configurable policy engine and an on-chain KYC/AML layer.

8.2 Core Components

Minter The central issuance contract. Whitelisted users deposit a baseAsset and receive an equal nominal amount of the corresponding synthetic token (HilBTC or HilUSD) via `mint`. The inverse, `redeem`, burns synthetic tokens and returns the baseAsset. The Minter holds collateral and periodically routes it to a custodian via `transferToCustody`. Yield is injected back into the system through `distributeYield`, which mints additional synthetic tokens credited to the StakingVault.

HilToken / **HilBTC** / **HilUSD** ERC-20 compliant synthetic tokens built on a shared `HilToken` base, minted and burned by the Minter and, in multi-chain deployments, by Chainlink CCT token pool contracts. Token transfers are subject to a compliance overlay that includes a Chainlink `PolicyEngine` check and a blacklist.

StakingVault An ERC-4626 vault where users deposit HilBTC or HilUSD to earn yield. Shares represent proportional ownership of the vault's growing asset base. Yield is credited by the Distributor and vests linearly over a configurable vesting period to smooth the per-share exchange rate. Withdrawals follow a two-phase flow: users call `redeem`/`withdraw` to start a cooldown (up to 90 days), after which assets held in escrow by `TokensHolder` can be claimed. An optional early exit lets users claim before the cooldown ends for a configurable fee (capped at 10%).

Distributor A yield-relay contract that sits between off-chain yield sources and the StakingVault. `distributeYield` reads the current yield amount from the StakingVault and instructs the Minter to mint the corresponding synthetic tokens directly into the vault, with a configurable minimum interval between distributions.

CompliantDepositRegistry Manages a registry of pre-approved deposit addresses for users bridging from traditional finance. Addresses are submitted in batches that enter a challenge period before being finalized. Once finalized, registered addresses can be claimed by users to establish a compliant on-chain presence.

ComplianceCheckerUpgradeable Implements SBT (Soulbound Token) based KYC/AML verification. An address is deemed compliant if it holds the SBTs required by at least one configured compliance option. The compliance options feed into the PolicyEngine used by the Minter's whitelist layer.

Supporting Helpers

- **Whitelist** / **PolicyEngine integration**: The `Whitelist` base contract (inherited by Minter) routes non-whitelisted callers through a Chainlink `PolicyEngine` for on-chain compliance checks.

- **BlacklistableUpgradeable**: A shared blacklist implementation used by both HilToken and StakingVault.
- **TokensHolder**: A simple escrow contract that holds users' underlying assets during the StakingVault cooldown period until they are claimed.

8.3 Cross-Chain Architecture (CCIP-CCT)

HilBTC and HilUSD are deployed as Chainlink Cross-Chain Tokens (CCT) using the `BurnAndMint` pool model. On the origin chain the Minter holds and manages collateral; on remote chains Chainlink CCIP `BurnMintTokenPool` contracts mint and burn the local HilToken deployment, avoiding liquidity fragmentation. This allows institutional users to hold and transfer synthetic Syntetika tokens across multiple EVM networks while collateral remains consolidated on the origin chain.

The StakingVault integration uses a `LockRelease` CCIP-CCT model: vault shares are locked on the source chain and released on the destination chain rather than burned and reminted, preserving the share accounting and yield-accrual state that lives exclusively on the origin chain.

9 Attestation

Cyfrin attests that it performed a security review of the Syntetika CCIP-CCT code provided by Syntetika.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-05-02-cyfrin-syntetika-ccip-cct-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Syntetika has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=6e6c570dce116ce48b09742ca6230b1c4a998ce3515e28831139b304d34a976e
F=2026-05-02-cyfrin-syntetika-ccip-cct-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.